QUESTIONS

- Q1 Composition in Solar-Powered Homes — 10 marks
- Q2 Function Overriding in Green Transport — 10 marks
- Q3 Virtual Base Class for Battery Data — 10 marks
- Q4 this Pointer in Carbon Monitoring — 10 marks
- Q5 Constructor Execution in Battery Recycling — 10 marks

5/5 answered

Q1 Composition in Solar-Powered Homes — 10 marks

A solar-powered house contains a BatteryStorage component. Analyze the use of composition and evaluate its advantage over inheritance.

Your Code

```
#include <iostream>
using namespace std;

class BatteryStorage {
    int capacity;

public:
    BatteryStorage(int c) {
        capacity = c;
    }

    void displayBattery() {
        cout << "Battery Capacity: " << capacity << " kWh" << endl;
    }
};

class SolarPoweredHouse {
    BatteryStorage battery;

public:
    SolarPoweredHouse(int capacity) : battery(capacity) {}

    void display() {
        cout << "Solar-Powered House" << endl;
        battery.displayBattery();
        cout << "Composition provides modularity and flexibility." << e
    }
};

int main() {
    int capacity;
    cin >> capacity;

    SolarPoweredHouse house(capacity);
    house.display();

    return 0;
}
```

Input

```
10
```

Output

Visible Test Cases

Test Case 1

QUESTIONS

- Q1 Composition in Solar-Powered Homes — 10 marks
- Q2 Function Overriding in Green Transport — 10 marks
- Q3 Virtual Base Class for Battery Data — 10 marks
- Q4 this Pointer in Carbon Monitoring — 10 marks
- Q5 Constructor Execution in Battery Recycling — 10 marks

5/5 answered

Q2 Function Overriding in Green Transport — 10 marks

A green-transport platform calculates environmental impact differently for ElectricBus and DieselBus. Analyze how function overriding improves runtime behavior.

Your Code

```cpp
#include <iostream>
using namespace std;

class Bus {
public:
    virtual void environmentalImpact() {
        cout << "Generic Bus: Environmental impact not defined." << end
    }
    virtual ~Bus() {}
};

class ElectricBus : public Bus {
public:
    void environmentalImpact() override {
        cout << "ElectricBus: Zero direct emissions. Very low environme
    }
};

class DieselBus : public Bus {
public:
    void environmentalImpact() override {
        cout << "DieselBus: Emits CO2 and pollutants. High environmenta
    }
};

int main() {
    Bus* buses[2];
    ElectricBus eBus;
    DieselBus dBus;

    buses[0] = &eBus;
    buses[1] = &dBus;

    for (int i = 0; i < 2; i++) {
        buses[i]->environmentalImpact();
    }

    return 0;
}
```

Input

```
stdin input
```

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

- Q1 ✓ Composition in Solar-Powered Homes — 10 marks
- Q2 ✓ Function Overriding in Green Transport — 10 marks
- Q3 ✓ Virtual Base Class for Battery Data — 10 marks
- Q4 ✓ this Pointer in Carbon Monitoring — 10 marks
- Q5 ✓ Constructor Execution in Battery Recycling — 10 marks

5/5 answered

Q3 Virtual Base Class for Battery Data

10 marks

An EV battery-management system inherits common battery information through multiple classes, causing duplicate data. Analyze the issue and resolve it using a virtual base class.

Your Code

```
#include <iostream>
#include <string>

class BatteryData {
public:
    std::string batteryID;
    int capacity;

    void inputBaseData() {
        std::cin >> batteryID >> capacity;
    }

    void displayBaseData() {
        std::cout << "Battery ID: " << batteryID << "\nCapacity: " << c
    }
};

class HealthMetrics : virtual public BatteryData {
public:
    int stateOfHealth;

    void inputHealth() {
        std::cin >> stateOfHealth;
    }

    void displayHealth() {
        std::cout << "State of Health: " << stateOfHealth << "%" << std
    }
};

class PerformanceMetrics : virtual public BatteryData {
public:
    double currentVoltage;

    void inputPerformance() {
        std::cin >> currentVoltage;
    }

    void displayPerformance() {
        std::cout << "Current Voltage: " << currentVoltage << "V" << st
    }
};

class BatteryManagementSystem : public HealthMetrics, public Performanc
public:
    void inputAllData() {
        inputBaseData();
        inputHealth();
        inputPerformance();
    }

    void displaySummary() {
        displayBaseData();
        displayHealth();
        displayPerformance();
    }
};

int main() {
    BatteryManagementSystem bms;
    bms.inputAllData();
    bms.displaySummary();
    return 0;
}
```

Input

```
BMS-99X 85 94 400.5
```

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

Q1 ✓
Composition in Solar-Powered Homes
10 marks

Q2 ✓
Function Overriding in Green Transport
10 marks

Q3 ✓
Virtual Base Class for Battery Data
10 marks

Q4 ✓
this Pointer in Carbon Monitoring
10 marks

Q5 ✓
Constructor Execution in Battery Recycling
10 marks

5/5 answered

Q4 this Pointer in Carbon Monitoring

10 marks

A carbon-monitoring system has a local variable and an instance variable with the same name. Analyze how the this pointer resolves the conflict.

Your Code

```cpp
#include <iostream>

class CarbonMonitor {
private:
    double carbonLevel;

public:
    void setCarbonLevel(double carbonLevel) {
        this->carbonLevel = carbonLevel;
    }

    void display() {
        std::cout << "Carbon Level: " << this->carbonLevel << " ppm" <<
    }
};

int main() {
    double inputLevel;
    if (std::cin >> inputLevel) {
        CarbonMonitor monitor;
        monitor.setCarbonLevel(inputLevel);
        monitor.display();
    }
    return 0;
}
```

Input

```
415.5
```

Output

Visible Test Cases

Test Case 1

QUESTIONS

Q1 ✓
Composition in Solar-Powered Homes
10 marks

Q2 ✓
Function Overriding in Green Transport
10 marks

Q3 ✓
Virtual Base Class for Battery Data
10 marks

Q4 ✓
this Pointer in Carbon Monitoring
10 marks

Q5 ✓
Constructor Execution in Battery Recycling
10 marks

5/5 answered

Q5 Constructor Execution in Battery Recycling

10 marks

A battery-recycling application initializes common Battery information before creating LithiumIonBattery objects. Analyze the execution order of base and derived class constructors.

Your Code

```
#include <iostream>
#include <string>

class Battery {
protected:
    std::string batteryType;

public:
    Battery(std::string type) : batteryType(type) {
        std::cout << "Base Class Constructor Executed: Initialising " <
    }
};

class LithiumIonBattery : public Battery {
private:
    int capacity;

public:
    LithiumIonBattery(std::string type, int cap) : Battery(type), capac
        std::cout << "Derived Class Constructor Executed: Initialising
    }
};

int main() {
    std::string type;
    int capacity;
    if (std::cin >> type >> capacity) {
        LithiumIonBattery li(type, capacity);
    }
    return 0;
}
```

Input

```
Recyclable 2500
```

Output

Visible Test Cases

Test Case 1